

Сравнительный анализ KAN и MLP в архитектуре TabKANet для табличных данных

Стеблий Мария

29.06.2026

- 1 Постановка задачи
 - Мотивация
 - Почему KAN?
- 2 Математические основы KAN
- 3 Архитектура TabKANet
 - Исходная архитектура
 - Модификация
- 4 Визуализация интерпретируемых компонентов
- 5 Эксперименты и результаты
 - Датасеты
 - Результаты
- 6 Выводы

Проблема

Табличные данные доминируют в бизнес-приложениях:

- Финансы
- Медицина
- Электронная коммерция

Основные вызовы нейросетевых подходов

- 1 Чувствительность к масштабу признаков
- 2 Отсутствие интерпретируемости
- 3 Сложность учета нелинейных взаимодействий

Почему Kolmogorov-Arnold Networks?

Ключевые преимущества KAN

- **Обучаемые функции на ребрах** вместо фиксированных активаций
- **Естественная интерпретируемость** через визуализацию сплайнов
- **Теоретическое обоснование:** аппроксимация любых непрерывных функций

Теорема Колмогорова–Арнольда

Theorem (Колмогоров–Арнольд)

Любая непрерывная функция $f : [0, 1]^n \rightarrow \mathbb{R}$ может быть представлена как:

$$f(x_1, \dots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$$

где $\phi_{q,p} : [0, 1] \rightarrow \mathbb{R}$ и $\Phi_q : \mathbb{R} \rightarrow \mathbb{R}$ — непрерывные функции.

Нейросетевая реализация

Глубокий KAN представляет собой композицию L слоев:

$$\text{KAN}(x) = (\Phi_{L-1} \circ \Phi_{L-2} \circ \dots \circ \Phi_0)(x)$$

Математическая формулировка KAN

Слой KAN

Каждый слой Φ_I отображает n_I -мерный вход в n_{I+1} -мерный выход:

$$\Phi_I(x) = \begin{pmatrix} \sum_{p=1}^{n_I} \phi_{I,1,p}(x_p) \\ \sum_{p=1}^{n_I} \phi_{I,2,p}(x_p) \\ \vdots \\ \sum_{p=1}^{n_I} \phi_{I,n_{I+1},p}(x_p) \end{pmatrix}$$

Обучаемая функция

$$\phi_{l,q,p}(x) = w_{l,q,p}^{(b)} \cdot \text{SiLU}(x) + w_{l,q,p}^{(s)} \cdot \text{spline}(x)$$

где spline — линейная комбинация B-сплайнов:

$$\text{spline}(x) = \sum_i c_{l,q,p,i} B_{i,k}(x)$$

KAN vs MLP: Сравнение

Характеристика	MLP	KAN
Активации на узлах	Фиксированные (ReLU, SiLU)	Отсутствуют
Обучаемые функции	На ребрах (веса)	На ребрах (сплайны)
Интерпретируемость	Низкая	Высокая
Аппроксимация	Универсальная	Универсальная
Количество параметров	Меньше	Больше

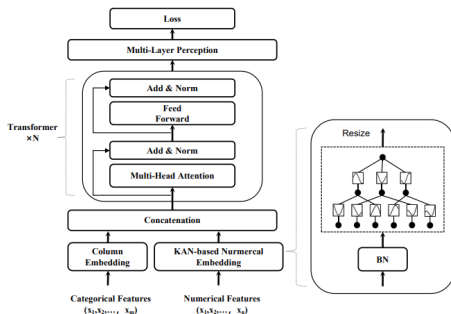
Ключевое отличие

KAN заменяет **фиксированные активации** на **обучаемые сплайны** на ребрах

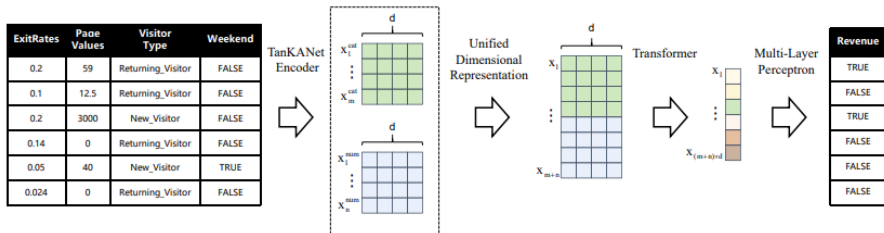
Архитектура TabKANet (оригинал)

Компоненты:

- 1 ColumnEmbedding (категориальные)
- 2 KAN + BatchNorm (непрерывные)
- 3 Transformer (Self-Attention + MLP FFN)
- 4 MLP Classifier



Архитектура TabKANet (оригинал)



Предложенная модификация: KAN-Transformer

Ключевая идея

Замена стандартного Transformer на **KAN-Transformer**, где Feed-Forward Network (FFN) заменена на KAN

Стандартный FFN:

KAN-FFN:

$$\text{FFN}_{\text{MLP}}(x) = W_2 \cdot \sigma(W_1 \cdot x + b_1) + b_2 \quad \text{FFN}_{\text{KAN}}(x) = (\Phi_{L-1} \circ \dots \circ \Phi_0)(x)$$

Преимущества

- Интерпретируемость FFN через визуализацию сплайнов
- Сохранение гладкости преобразований
- Адаптивность к распределению данных

Формула модели

$$\text{Emb}_{\text{cat}} = \text{ColumnEmbedding}(X_{\text{cat}})$$

$$\text{Emb}_{\text{cont}} = \text{BatchNorm}(\text{KAN}(X_{\text{cont}}))$$

$$X = [\text{Emb}_{\text{cat}}; \text{Emb}_{\text{cont}}]$$

For $l = 1$ to L :

$$X = X + \text{Attention}(X)$$

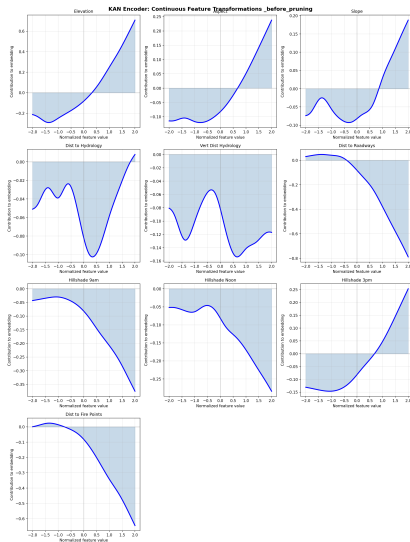
$$X = X + \text{KAN}(X) \quad (\text{вместо MLP FFN})$$

$$\text{Output} = \text{MLP}(X)$$

KAN Encoder: Влияние непрерывных признаков

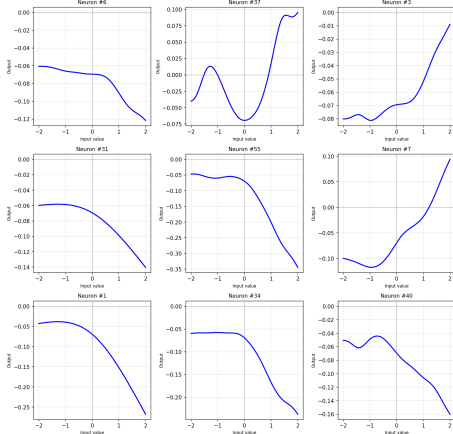
KAN Encoder

- По оси X — нормализованное значение признака
- По оси Y — вклад в эмбединг
- Форма кривой показывает характер влияния признака



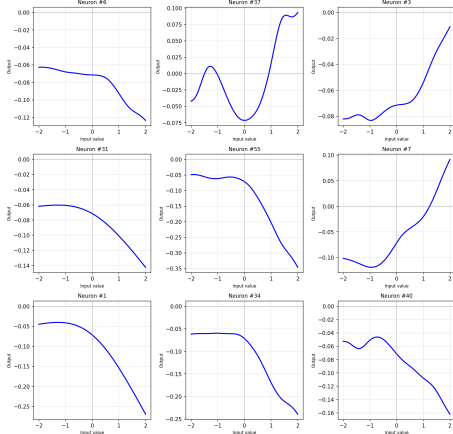
KAN-Transformer FFN: Слой 1

KAN-Transformer Layer 1: FFN Splines _before_pruning



До прунинга

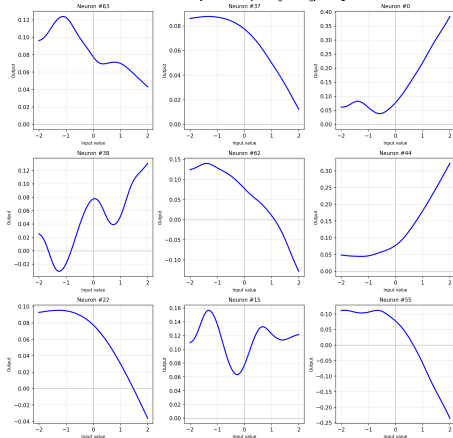
KAN-Transformer Layer 1: FFN Splines _after_pruning



После прунинга (20%)

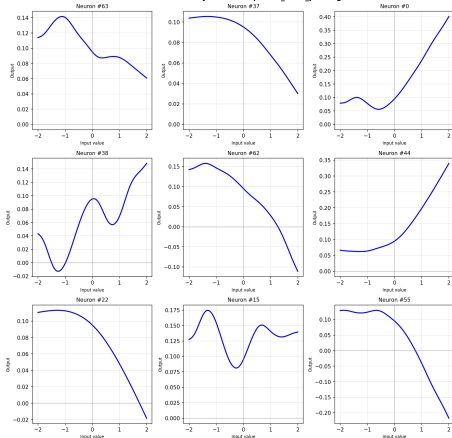
KAN-Transformer FFN: Слой 2

KAN-Transformer Layer 2: FFN Splines _before_pruning



До прунинга

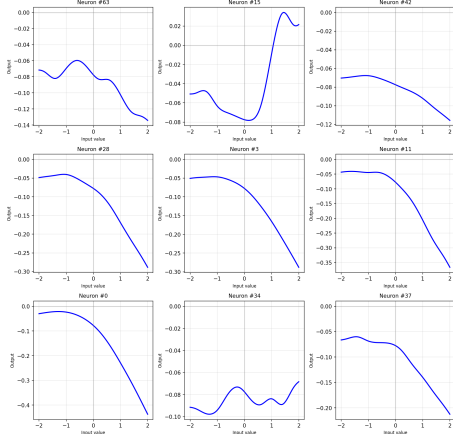
KAN-Transformer Layer 2: FFN Splines _after_pruning



После прунинга (20%)

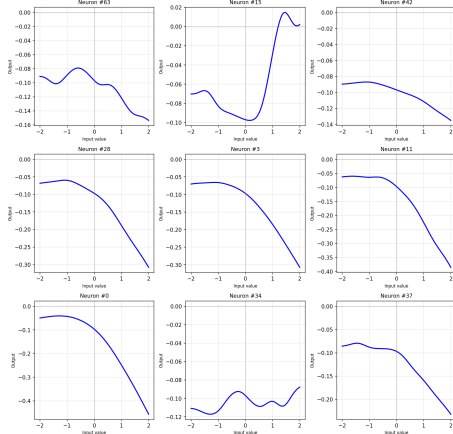
KAN-Transformer FFN: Слой 3

KAN-Transformer Layer 3: FFN Splines _before_pruning



До прунинга

KAN-Transformer Layer 3: FFN Splines _after_pruning



После прунинга (20%)

Влияние прунинга на FFN сплайны: итоги

Ключевые наблюдения

- ❶ После прунинга количество активных нейронов уменьшилось на каждом слое
- ❷ Оставшиеся сплайны сохранили основные формы
- ❸ **Слой 1:** синусоидальные формы — поиск периодических паттернов
- ❹ **Слой 2:** экспоненциальные формы с колебаниями — комбинирование
- ❺ **Слой 3:** гладкие экспоненциальные кривые — финальная функция

Вывод

Прунинг удаляет избыточные нейроны, не нарушая интерпретируемость KAN.

Использованные датасеты

Датасет	Тип	Признаки	Размер
BankMarketing	Бинарная классификация	17	45 211
OnlineShoppers	Бинарная классификация	18	12 330
ImageSegmentation	Мультикласс (7)	20	2 310
ForestCovertype	Мультикласс (7)	55	581 012
CA House	Регрессия	10	20 640
CPU Small	Регрессия	13	8 192

Метрики

- Бинарная классификация: **AUC**
- Мультиклассовая: **Macro F1**
- Регрессия: **RMSE**

Результаты: Бинарная классификация

Модель. BankMarketing (AUC)	Mean	Std	Δ
KAN-Transformer (10 эпох)	0.9331	0.0011	—
KAN-Transformer (fine-tuned)	0.9345	0.0010	+0.0014
Standard Transformer (fine-tuned)	0.9327	0.0020	+0.0020

Модель. OnlineShoppers (AUC)	Mean	Std	Δ
KAN-Transformer (10 эпох)	0.8916	0.0006	—
KAN-Transformer (fine-tuned)	0.8934	0.0006	+0.0018
Standard Transformer (fine-tuned)	0.8923	0.0007	+0.0010

Вывод: KAN-Transformer стабильнее (меньший std) и показывает лучшие результаты.

Результаты: Мультиклассовая классификация

Модель. ImageSegmentation	Mean	Std	Δ
KAN-Transformer (10 эпох)	0.9356	0.0137	—
KAN-Transformer (fine-tuned)	0.9476	0.0172	+0.0120
Standard Transformer (fine-tuned)	0.9623	0.0061	+0.0073

Модель. ForestCovertime	Mean	Std	Δ
KAN-Transformer (10 эпох)	0.6537	0.0086	—
KAN-Transformer (fine-tuned)	0.6915	0.0054	+0.0378
Standard Transformer (fine-tuned)	0.6776	0.0046	+0.0296

Вывод: На большом датасете KAN показывает лучшее восстановление после прунинга.

Результаты: Регрессия

Модель. CA House (RMSE)	Mean	Std	Δ
KAN-Transformer (10 эпох)	7.249	0.384	—
KAN-Transformer (fine-tuned)	6.699	0.383	-0.550
Standard Transformer (fine-tuned)	6.713	0.102	-0.560

Модель. CPU Small (RMSE)	Mean	Std	Δ
KAN-Transformer (10 эпох)	70.17	1.124	—
KAN-Transformer (fine-tuned)	69.26	1.159	-0.91
Standard Transformer (fine-tuned)	68.85	1.288	-0.90

Вывод: KAN-Transformer не уступает стандартному Transformer в регрессионных задачах.

Анализ эффективности прунинга (20% нейронов)

Датасет	KAN: падение	KAN: восст.	Std: восст.
BankMarketing	-0.0003	+0.0014	+0.0020
OnlineShoppers	-0.0000	+0.0009	+0.0005
ImageSegmentation	-0.0012	+0.0121	+0.0073
ForestCovertime	-0.0091	+0.0378	+0.0296
Cahouse	0.0188	-0.5499	-0.5601
CPU Small	0.0034	-0.9154	-0.8937

Ключевое наблюдение

KAN демонстрирует лучшее восстановление после прунинга на сложных мультиклассовых датасетах (ForestCovertime: +3.78% против +2.96% у стандартного Transformer).

- 1 **KAN-Transformer не уступает стандартному Transformer по качеству, а на некоторых датасетах превосходит его.**
- 2 **Прунинг 20% нейронов практически не влияет на качество (падение менее 0.01 по AUC/F1 для большинства датасетов).**
- 3 **KAN демонстрирует лучшее восстановление после прунинга на сложных мультиклассовых задачах (+3.78% против +2.96%).**
- 4 **KAN обеспечивает интерпретируемость, недоступную для MLP: визуализация сплайнов позволяет понять характер влияния признаков.**

Преимущества предложенной модификации

KAN-Transformer

- Визуализация FFN-преобразований
- Понимание внутренних процессов
- Отладка и интерпретация

Сохранение производительности

- Сравнимые метрики с MLP
- Лучшая стабильность
- Устойчивость к прунингу

Направления дальнейших исследований

- 1 Сравнение с градиентным бустингом (XGBoost, CatBoost) на табличных данных
- 2 Исследование влияния размера сетки сплайнов на качество и интерпретируемость
- 3 Применение KAN к категориальным признакам через пост-обработку эмбедингов
- 4 Комбинирование KAN с attention-механизмами для улучшения интерпретации



Weihaio Gao, Zheng Gong, Zhuo Deng, Fujun Rong, Chucheng Chen, Lan Ma.

TabKANet: Tabular Data Modeling with Kolmogorov-Arnold Network and Transformer.

arXiv preprint arXiv:2409.08806, 2024.

<https://arxiv.org/abs/2409.08806>.



Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, M. Tegmark.

KAN: Kolmogorov-Arnold Networks.

arXiv preprint arXiv:2404.19756, 2024.



Ashish Vaswani, Noam Shazeer, Niki Parmar, et al.

Attention Is All You Need.

Advances in Neural Information Processing Systems, 2017.